


**Laboratorium**
**Robotika dan Sistem Cerdas**
**Departemen Teknik Komputer**
***Institut Teknologi Sepuluh Nopember***

# Laporan Akhir
# Praktikum Sistem Mikroprosesor dan Sistem Tertanam

## Pengantar Mikrokontroler

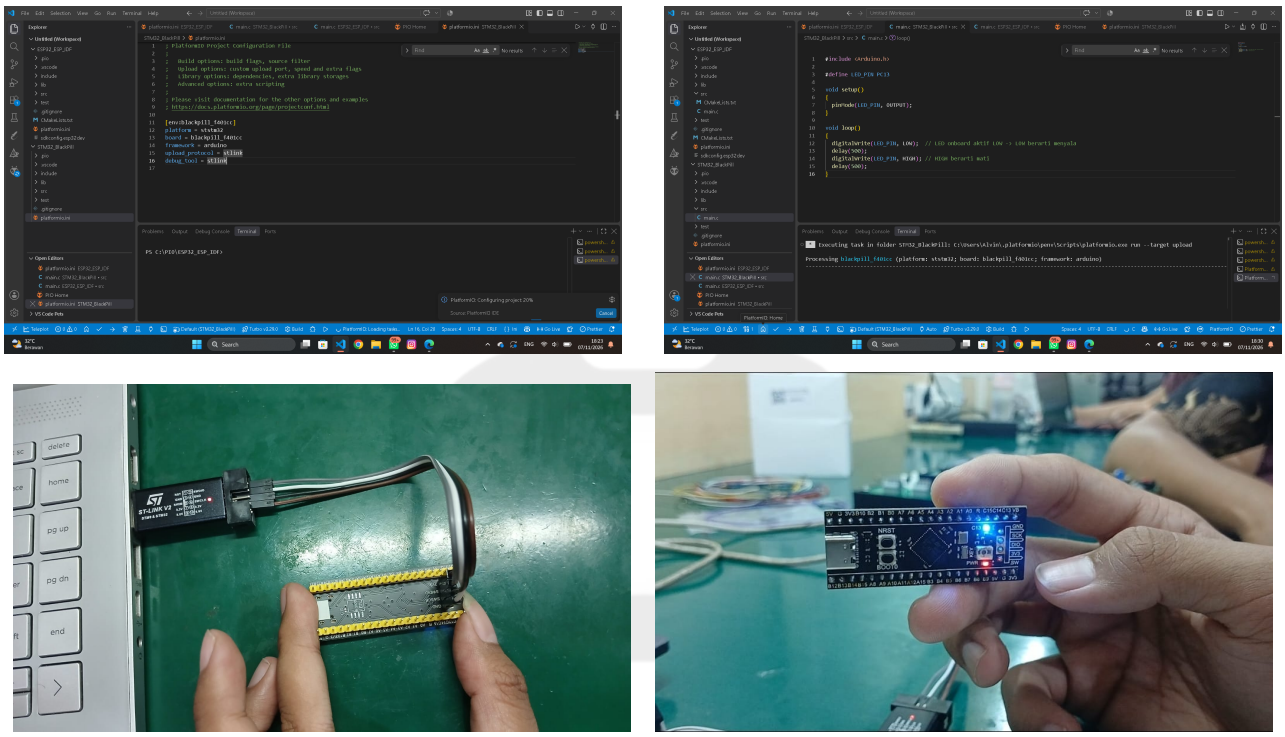
Muhammad Alvin Nafisal Abror - 5024241098

2026

# 1 Langkah-Langkah Percobaan

## 1.1 Percobaan 1 Pengenalan STM32 Blackpill (Blink)

Pada percobaan pertama dilakukan pengenalan board STM32 Blackpill menggunakan framework Arduino melalui PlatformIO pada Visual Studio Code. Pengujian dilakukan dengan menghubungkan board STM32 ke ST-Link menggunakan komunikasi SWD. Program Blink dibuat untuk mengatur LED onboard pada pin PC13 agar menyala dan mati secara bergantian dengan interval 500 ms. Selanjutnya, program dikompilasi dan diunggah ke board untuk memastikan mikrokontroler dapat menjalankan instruksi dengan baik.



**Gambar 1:** Dokumentasi Percobaan 1

## 1.2 Percobaan 2 Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

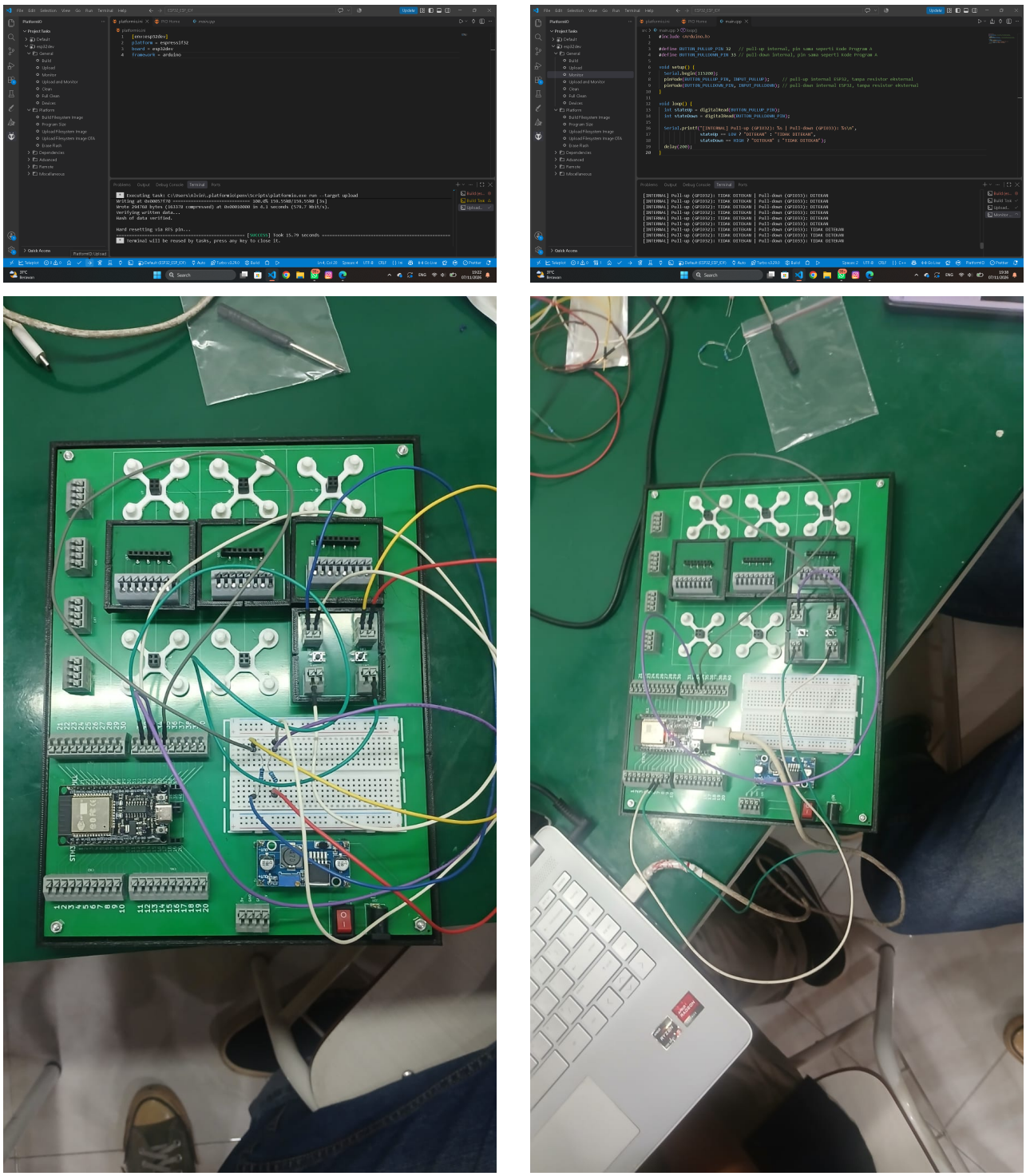
Pada percobaan kedua dilakukan pengenalan board ESP32 DevKit menggunakan framework ESP-IDF. Konfigurasi project PlatformIO disesuaikan dengan board ESP32 dan framework yang digunakan. Program Blink dibuat menggunakan fungsi `app_main()` dengan GPIO 2 sebagai output LED. Setelah program berhasil dikompilasi dan diunggah, dilakukan pengamatan terhadap LED onboard yang menyala dan mati secara bergantian setiap 500 ms. Percobaan ini juga bertujuan memahami perbedaan struktur pemrograman antara framework Arduino dan ESP-IDF.



**Gambar 2:** Dokumentasi Percobaan 2

## 1.3 Percobaan 3 Pull-up dan Pull-down Eksternal vs Internal

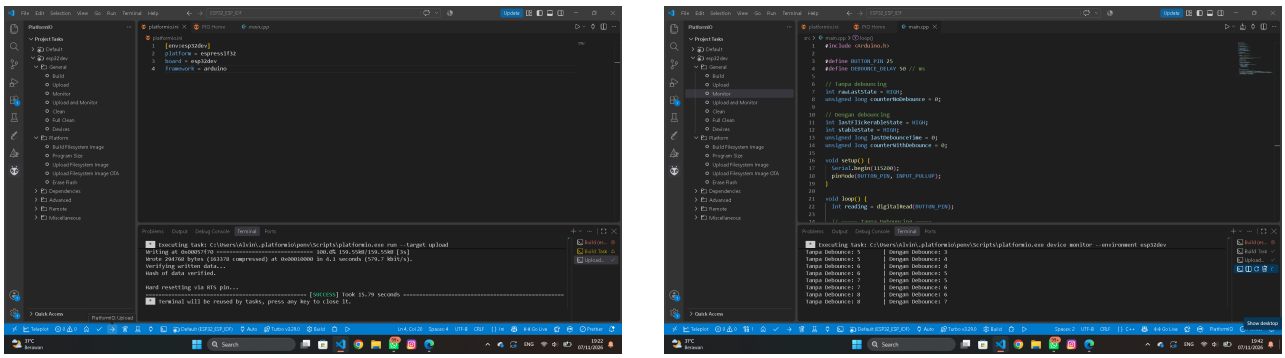
Pada percobaan ketiga dilakukan pengujian konfigurasi pull-up dan pull-down pada pushbutton menggunakan ESP32. Pengujian pertama menggunakan resistor eksternal 10k$\Omega$ yang dipasang pada GPIO 32 dan GPIO 33. Selanjutnya, resistor eksternal dilepas dan pengujian dilakukan kembali menggunakan fitur pull-up dan pull-down internal ESP32. Perubahan kondisi tombol diamati melalui Serial Monitor untuk membandingkan hasil pembacaan antara konfigurasi eksternal dan internal, baik ketika tombol ditekan maupun tidak ditekan.

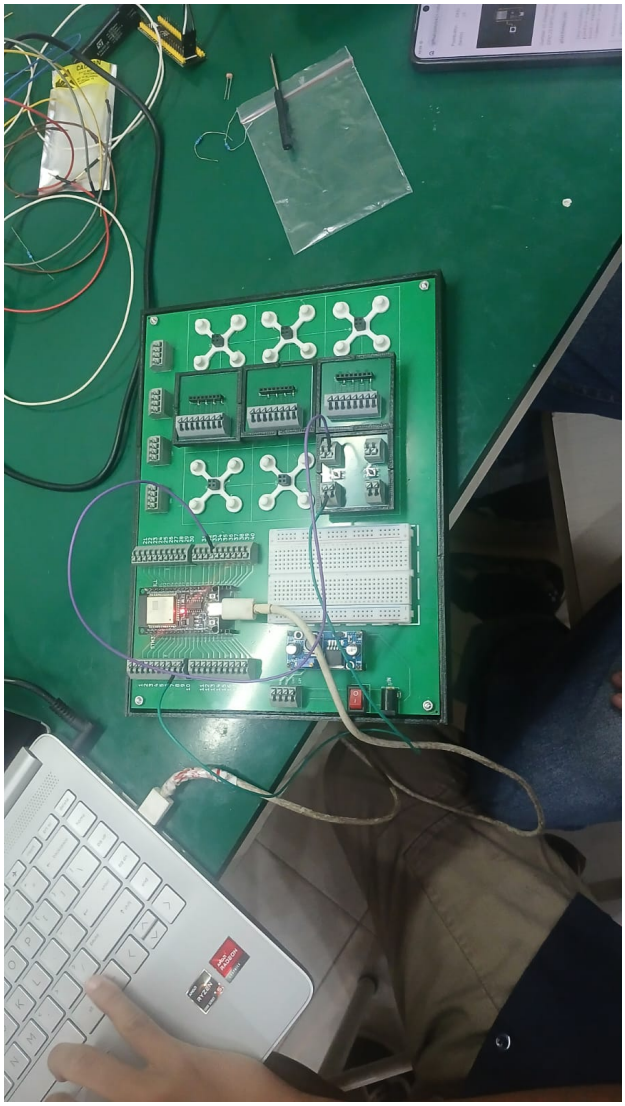


**Gambar 3:** Dokumentasi Percobaan 3

## 1.4 Percobaan 4 Debouncing pada Input Tombol GPIO

Pada percobaan keempat dilakukan pengujian debouncing untuk mengatasi perubahan sinyal yang tidak stabil akibat getaran kontak mekanik pada pushbutton. Tombol dihubungkan ke GPIO 25 ESP32 dengan konfigurasi pull-up internal. Program dibuat untuk membandingkan jumlah pembacaan tombol tanpa debouncing dan dengan debouncing menggunakan metode software. Pengujian dilakukan dengan mengubah nilai waktu debounce menjadi 10 ms, 50 ms, dan 200 ms. Hasil setiap pengujian diamati melalui Serial Monitor untuk mengetahui pengaruh perbedaan waktu debounce terhadap kestabilan pembacaan tombol.

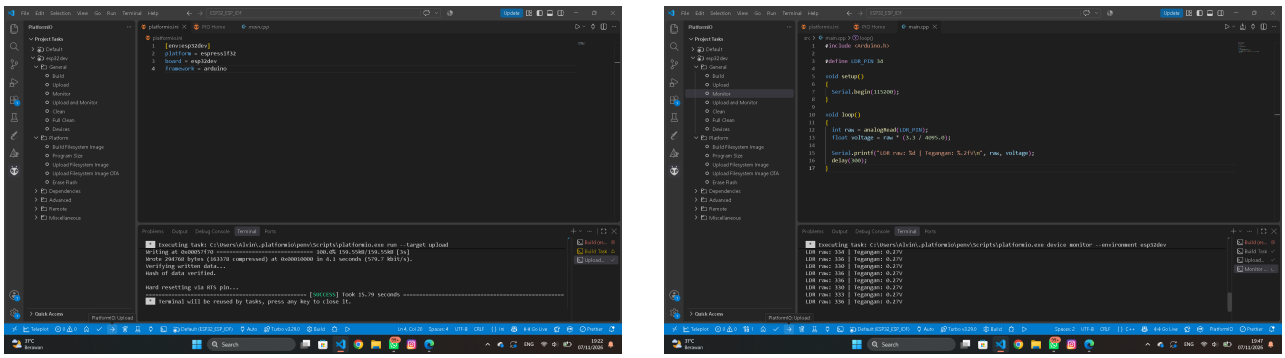


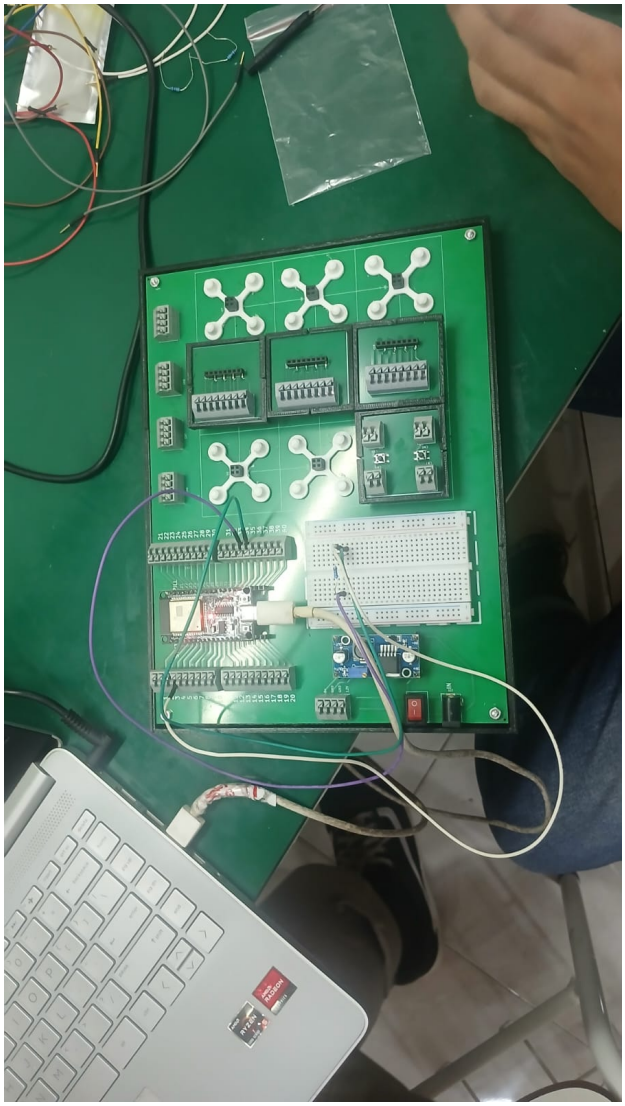


**Gambar 4:** Dokumentasi Percobaan 4

## 1.5 Percobaan 5 Akses GPIO Analog: Pembacaan ADC dengan LDR

Pada percobaan kelima dilakukan pengujian pembacaan input analog menggunakan sensor LDR (*Light Dependent Resistor*) pada ESP32. Sensor LDR dirangkai bersama resistor 10k$\Omega$ sebagai pembagi tegangan, kemudian titik tengah rangkaian dihubungkan ke GPIO 34 sebagai input ADC. Program dibuat untuk membaca nilai analog menggunakan fungsi `analogRead()` dan mengonversinya menjadi nilai tegangan. Pengujian dilakukan dengan memberikan kondisi pencahayaan yang berbeda, yaitu menutup permukaan LDR dan mengarahkannya ke sumber cahaya. Perubahan nilai ADC dan tegangan diamati melalui Serial Monitor untuk mengetahui respons sensor terhadap intensitas cahaya.





**Gambar 5:** Dokumentasi Percobaan 5

## 2 Analisis Hasil Percobaan

Pada percobaan pertama, pengujian dilakukan menggunakan board STM32 Blackpill dengan program Blink melalui framework Arduino. Berdasarkan teori, pin GPIO yang dikonfigurasi sebagai output dapat digunakan untuk mengendalikan kondisi LED melalui fungsi `digitalWrite()`. Hasil yang diharapkan adalah LED onboard pada pin PC13 menyala dan mati secara bergantian dengan interval 500 ms. Karena LED pada STM32 Blackpill menggunakan konfigurasi aktif LOW, kondisi LOW menyebabkan LED menyala, sedangkan HIGH menyebabkan LED mati. Keberhasilan percobaan

menunjukkan bahwa konfigurasi PlatformIO, komunikasi ST-Link, dan proses flashing program telah berjalan dengan baik. Apabila LED tidak bekerja sesuai instruksi, kemungkinan penyebabnya adalah kesalahan koneksi SWDIO atau SWCLK, pemilihan konfigurasi board, maupun kesalahan proses upload.

Pada percobaan kedua, ESP32 digunakan untuk menjalankan program Blink menggunakan framework ESP-IDF. Berbeda dengan framework Arduino yang menggunakan fungsi `setup()` dan `loop()`, ESP-IDF menggunakan fungsi `app_main()` sebagai titik awal program. Pengaturan GPIO dilakukan melalui fungsi `gpio_set_direction()` dan `gpio_set_level()`, sedangkan pengaturan waktu menggunakan `vTaskDelay()`. Hasil yang diharapkan adalah LED pada GPIO 2 menyala dan mati secara bergantian dengan interval 500 ms. Hal ini menunjukkan bahwa ESP32 dapat menjalankan instruksi output digital melalui framework ESP-IDF. Apabila LED tidak menyala, faktor yang perlu diperiksa meliputi konfigurasi project, pemilihan GPIO, koneksi USB, serta kesesuaian pin LED dengan jenis board yang digunakan.

Pada percobaan ketiga, dilakukan perbandingan pembacaan pushbutton menggunakan konfigurasi pull-up dan pull-down eksternal serta internal. Berdasarkan teori, konfigurasi pull-up menghasilkan kondisi HIGH ketika tombol tidak ditekan dan LOW ketika tombol ditekan. Sebaliknya, konfigurasi pull-down menghasilkan kondisi LOW ketika tombol tidak ditekan dan HIGH ketika tombol ditekan. Hasil pengujian yang diharapkan adalah kedua konfigurasi memberikan pembacaan logika yang sama meskipun menggunakan metode pemasangan resistor yang berbeda. Penggunaan resistor berfungsi menjaga kondisi GPIO agar tidak mengalami floating. Apabila hasil pembacaan tidak sesuai teori, kemungkinan penyebabnya adalah kesalahan pemasangan resistor, orientasi kaki pushbutton, koneksi GND, atau resistor eksternal yang belum dilepas ketika menggunakan konfigurasi internal.

Pada percobaan keempat, pengujian dilakukan untuk mengetahui pengaruh debouncing terhadap pembacaan input pushbutton. Berdasarkan teori, kontak mekanik pada tombol dapat menghasilkan perubahan sinyal berulang dalam waktu singkat atau disebut bouncing. Kondisi tersebut dapat menyebabkan satu kali penekanan tombol terbaca sebagai beberapa input. Melalui program yang dibuat, pembacaan tanpa debouncing dibandingkan dengan pembacaan menggunakan penyaringan waktu. Pengujian dilakukan dengan variasi waktu debounce 10 ms, 50 ms, dan 200 ms. Semakin besar waktu debounce, semakin lama sinyal harus stabil sebelum diterima sebagai input yang valid, sehingga respons tombol dapat menjadi lebih lambat. Hasil percobaan digunakan untuk memahami bahwa pemilihan waktu debounce perlu mempertimbangkan kestabilan pembacaan dan kecepatan respons tombol. Perbedaan hasil dapat dipengaruhi oleh kondisi fisik pushbutton, kecepatan penekanan, serta nilai waktu debounce yang digunakan.

Pada percobaan kelima, dilakukan pengujian pembacaan intensitas cahaya menggunakan sensor LDR yang dihubungkan ke GPIO 34 ESP32. Berdasarkan teori, resistansi LDR berubah mengikuti intensitas cahaya yang diterima. Perubahan resistansi tersebut menghasilkan perubahan tegangan pada rangkaian pembagi tegangan yang kemudian dibaca oleh ADC. ESP32 mengubah tegangan analog menjadi nilai digital dengan rentang pembacaan 0–4095. Hasil yang diharapkan adalah nilai ADC dan tegangan mengalami perubahan ketika sensor berada pada kondisi pencahayaan yang berbeda. Nilai pembacaan tersebut menunjukkan bahwa GPIO tidak hanya dapat digunakan sebagai input dan output digital, tetapi juga sebagai input analog. Apabila hasil pembacaan tidak stabil, faktor yang dapat memengaruhi antara lain perubahan intensitas cahaya lingkungan, posisi sumber cahaya, pemasangan resistor, koneksi rangkaian, serta karakteristik sensor LDR.

# 3 Hasil Tugas Modul

1. **Analisis mengapa debouncing tetap diperlukan pada rangkaian matrix pushbutton ini, walaupun pada satu waktu hanya satu tombol yang biasanya ditekan, kaitkan dengan penyebab bouncing pada C.5 Debouncing.**

   Debouncing tetap diperlukan karena kontak mekanis pada pushbutton dapat mengalami getaran atau pantulan (*bouncing*) ketika ditekan maupun dilepaskan. Kondisi tersebut menyebabkan sinyal digital berubah secara cepat antara HIGH dan LOW sebelum mencapai keadaan stabil. Meskipun hanya satu tombol yang ditekan, bouncing dapat menyebabkan ESP32 membaca beberapa perubahan sinyal seolah-olah terjadi penekanan berulang. Akibatnya, LED dapat berubah status lebih dari satu kali. Oleh karena itu, debouncing diterapkan dengan memberikan waktu tunggu tertentu, misalnya 50 ms, agar perubahan status tombol hanya diproses setelah pembacaannya stabil.

2. **Analisis konfigurasi pull-up atau pull-down (sesuai rancangan Anda) yang digunakan pada pembacaan kolom tombol, apa kondisi (HIGH/LOW) pin tersebut saat tombol tidak ditekan, dan mengapa kondisi tersebut tidak boleh dibiarkan floating?**

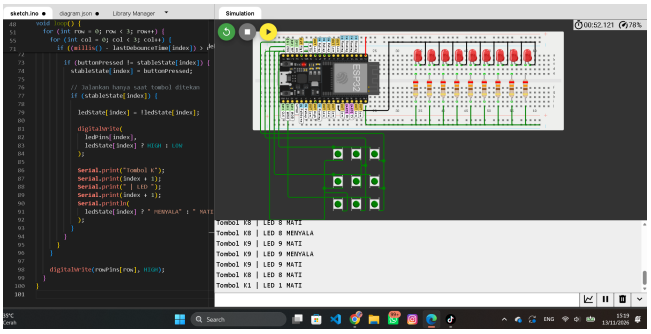
   Pada rancangan ini digunakan konfigurasi *internal pull-up* dengan mengatur pin kolom sebagai `INPUT_PULLUP`. Ketika tombol tidak ditekan, pin kolom berada pada kondisi HIGH karena terhubung secara internal ke tegangan 3,3 V melalui resistor pull-up. Ketika tombol ditekan dan baris yang bersangkutan diaktifkan LOW, pin kolom akan terbaca LOW.

   Kondisi pin tidak boleh dibiarkan *floating* karena tegangan input menjadi tidak pasti dan rentan terhadap gangguan listrik di sekitarnya. Hal tersebut dapat menyebabkan ESP32 membaca perubahan HIGH dan LOW secara tidak menentu, meskipun tombol tidak ditekan. Penggunaan resistor pull-up memberikan kondisi logika yang stabil sehingga pembacaan tombol lebih akurat.

3. **Andaikan salah satu tombol pada matrix ini digantikan dengan LDR yang dibaca melalui ADC (seperti Percobaan 5), analisis perbedaan mendasar antara membaca GPIO sebagai digital input (tombol, hanya HIGH/LOW) dan sebagai analog input (ADC), mengapa ADC dapat merepresentasikan lebih dari dua kondisi?**

   Pembacaan digital input hanya mengenali dua kondisi logika, yaitu HIGH dan LOW. Pada rangkaian pushbutton, kedua kondisi tersebut digunakan untuk menunjukkan apakah tombol sedang ditekan atau tidak. Sementara itu, pembacaan analog menggunakan ADC (*Analog-to-Digital Converter*) untuk mengubah tegangan masukan menjadi nilai digital dalam rentang tertentu.

   ADC dapat merepresentasikan lebih dari dua kondisi karena mampu membagi rentang tegangan masukan menjadi beberapa tingkat pembacaan. Pada ESP32 dengan ADC 12-bit, hasil konversi memiliki rentang 0–4095. Apabila LDR digunakan sebagai sensor, perubahan intensitas cahaya akan mengubah resistansi dan tegangan keluarannya sehingga menghasilkan nilai ADC yang bervariasi. Dengan demikian, mikrokontroler dapat membedakan berbagai tingkat intensitas cahaya, bukan hanya kondisi HIGH dan LOW.



**Gambar 6:** ss kode dan rangkaian, karena banyaknya error pada rangkaian saya hanya melampirkan 1 dokumentasi hasil

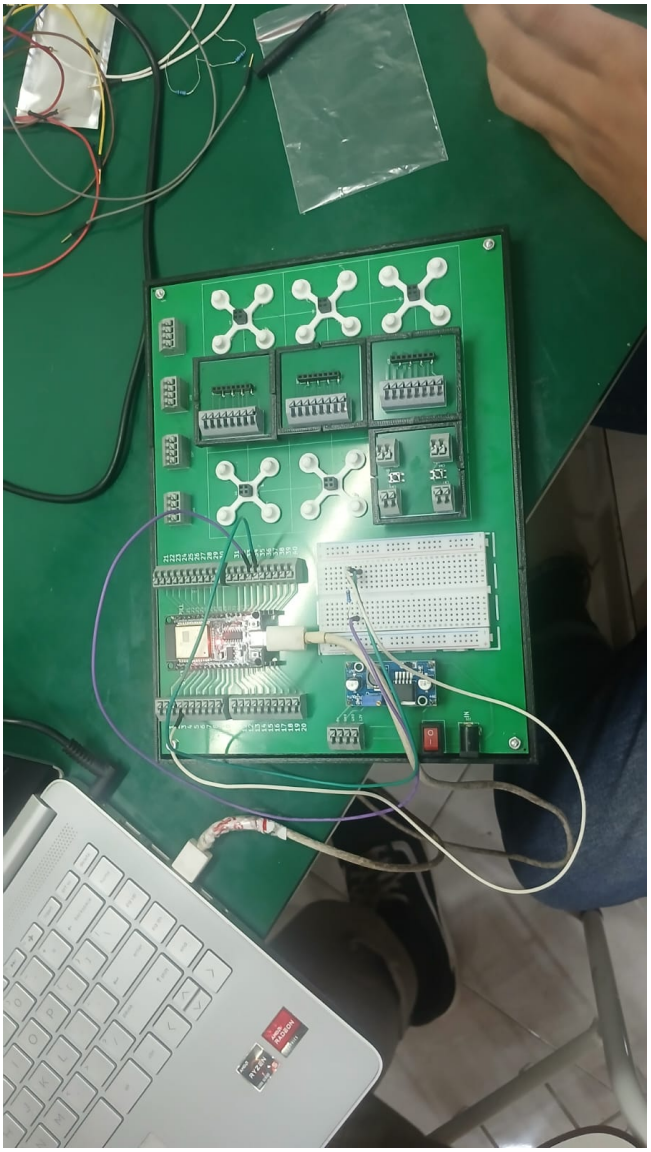
# 4 Kesimpulan

Berdasarkan praktikum Modul 1 tentang Pengantar Mikrokontroler, dapat disimpulkan bahwa board STM32 Blackpill dan ESP32 dapat diprogram menggunakan PlatformIO dengan framework yang berbeda, yaitu Arduino dan ESP-IDF. Pengujian LED Blink memperkenalkan penggunaan GPIO sebagai output digital, sedangkan percobaan pushbutton menunjukkan penerapan konfigurasi pull-up dan pull-down eksternal maupun internal untuk membaca input digital secara stabil.

Selain itu, penerapan debouncing memberikan pemahaman mengenai pentingnya penyaringan sinyal tombol untuk menghindari pembacaan ganda akibat bouncing. Penggunaan sensor LDR juga menunjukkan bahwa GPIO dapat dimanfaatkan sebagai input analog melalui ADC untuk membaca perubahan intensitas cahaya. Secara keseluruhan, praktikum ini memberikan pemahaman dasar mengenai konfigurasi mikrokontroler, pemrograman GPIO, pembacaan input digital, serta pengolahan sinyal analog sebagai dasar pengembangan sistem embedded.

# 5 Lampiran

## 5.1 Dokumentasi saat praktikum



**Gambar 7:** Caption